

Auditoria do código e painel multi-algoritmo

- Auditoria do código e painel multi-algoritmo
 - Parte I — Auditoria: as inferências estão coerentes com o modelo?
 - Parte II — Seleção de algoritmo
 - Parte III — Como funciona cada algoritmo
 - Parte IV — A plataforma nova

Auditoria do código e painel multi-algoritmo

Parte I — Auditoria: as inferências estão coerentes com o modelo?

A matemática está correta; a interface é que discorda do modelo. A reimplementação em TypeScript (padronização → produto escalar → sigmoide → interpolação isotônica) reproduz o scikit-learn, e o `modelos.json` embarcado tem MD5 idêntico ao exportado do treino. Os problemas estão na camada de decisão da interface.

1. O padrão contradiz a evidência do próprio painel

`src/main.ts` define `modeloEscolhido = "genes"` e marca esse conjunto como padrão. A nota exibida logo abaixo diz: “*este modelo discrimina pior que o clínico (AUC 0,639 contra 0,719)*”. O produto entrega por padrão o modelo que ele mesmo declara inferior.

2. O veredito de confiabilidade pertence a outro modelo

As barras de AUC e IC 95% vêm de `confiabilidade_por_subtipo`, estimada **sobre o modelo combinado calibrado**. A interface as exibe qualquer que seja o conjunto selecionado. O usuário lê “AUC 0,678 em LumA” supondo que descreve a predição na tela.

3. Predição confiante a partir de nada

Com o conjunto de genes como padrão e nenhuma expressão carregada, todos os genes assumem $z = 0$ e a tela exibe **40,4%** já na primeira carga — número calculado sobre um paciente médio que não existe.

4. Campo clínico em branco vira zero

`montaVetor`: `if (n in cl) return isNaN(cl[n]) ? 0 : cl[n];`. Zero não é “ausente”: é idade zero. Simulado no modelo combinado, com idade em branco o risco cai de **40,3% para 15,7%**. Hoje há uma checagem em `calcula()` que contém o caso, mas o fallback continua no código.

5. Subtipo indefinido cai silenciosamente em LumA

`if (!sub) sub = "LumA"` — sem expressão e sem seleção, o app assume o subtipo mais frequente, usa `sub_LumA = 1` no modelo e mostra o veredito do LumA, sem qualquer aviso.

6. Afirmação factualmente falsa

A mensagem do botão de predição informa “*106 genes selecionados + PAM50*”. O modelo `genes` tem 106 variáveis e **nenhuma** delas é dummy de subtipo. Verificado no JSON.

7. Perdas e inconsistências de empacotamento

O `_headers` com a CSP restritiva não veio no pacote — perde-se `connect-src 'none'`, que era a garantia técnica de que nada trafega. O `metadata.json` declara `MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API`, embora nenhuma chamada externa exista no código. O painel de estágio é exibido sem que haja campo de entrada de estágio, então o veredito nunca reage a ele — apesar de III-IV ser justamente um estrato não confiável.

Correções aplicadas na nova versão

Problema	Correção
Padrão incoerente	Conjunto clínico e algoritmo de maior AUC como padrão
Veredito de outro modelo	O texto do veredito nomeia o modelo em uso e adverte que as barras se referem ao combinado
Predição a partir de nada	Sem expressão, o conjunto genômico retorna “ indisponível ” com explicação, em vez de um número
NaN → 0	Campo em branco bloqueia o modelo que depende dele; nunca vira zero
LumA silencioso	Subtipo passa a ser obrigatório onde entra no modelo
“+ PAM50”	Texto corrigido; cada modelo exibe a contagem real de variáveis
CSP ausente	<code>_headers</code> reincluído

Parte II — Seleção de algoritmo

Dezesseis modelos: seis algoritmos × três conjuntos de variáveis (o gradient boosting clássico ficou restrito ao conjunto clínico — em 489 colunas o custo é proibitivo, e o XGBoost representa a família de boosting). Hiperparâmetros por busca aleatória **dentro** da validação cruzada; IC 95% pelo método de DeLong.

Conjunto	Algoritmo	Var.	AUC	IC 95%	Brier	H-L
Combinado	XGBoost	500	0,7488	0,725–0,773	0,198	0,999
Clínico	Rede neural (MLP)	11	0,7379	0,713–0,762	0,202	0,95
Clínico	XGBoost	11	0,7352	0,711–0,760	0,203	0,43
Clínico	Random Forest	11	0,7334	0,709–0,758	0,203	0,99
Clínico	Gradient Boosting	11	0,7328	0,708–0,758	0,204	0,52
Combinado	Extra Trees	500	0,7253	0,701–0,750	0,206	0,998
Clínico	Regressão logística	11	0,7190	0,694–0,744	0,208	0,65
Clínico	Extra Trees	11	0,7182	0,693–0,743	0,208	0,94
Combinado	Random Forest	500	0,7047	0,679–0,730	0,213	0,84
Combinado	Regressão logística	500	0,6914	0,665–0,717	0,217	0,26
Combinado	Rede neural (MLP)	500	0,6702	0,644–0,697	0,224	0,97
Genes	XGBoost	489	0,6632	0,636–0,690	0,223	0,99
Genes	Random Forest	489	0,6511	0,624–0,678	0,226	0,88
Genes	Extra Trees	489	0,6488	0,622–0,676	0,227	0,996
Genes	Regressão logística	489	0,6314	0,604–0,659	0,231	0,83
Genes	Rede neural (MLP)	489	0,6194	0,592–0,647	0,234	0,94

O que mudou na conclusão anterior

O **XGBoost sobre o conjunto combinado (0,7488)** é o primeiro modelo a superar o melhor puramente clínico. Isso qualifica — sem anular — o resultado que eu vinha reportando: a regressão logística não extraía nada dos genes além do clínico, mas um método capaz de capturar interações extrai **algum** sinal. Duas ressalvas honestas:

- O IC 95% (0,725–0,773) se sobrepõe amplamente ao do MLP clínico (0,713–0,762). A diferença de 1,1 ponto de AUC **não é conclusiva**.
- O ganho custa 489 variáveis adicionais. Em termos de custo por informação, continua sendo um péssimo negócio: 11 variáveis de rotina entregam 0,738.

Sobre “otimizar ao máximo a precisão”

Os treze modelos entre 0,69 e 0,75 têm intervalos que se sobrepõem quase inteiramente. O teto de ~0,75 não cede a troca de algoritmo nem a busca de hiperparâmetros — é limite da informação contida nestes dados, e a busca aninhada já foi feita. O que ainda pode elevar a precisão de verdade: validação externa (TCGA-BRCA), restrição aos luminais em estágio II, modelos de risco competitivo e Random Survival Forest, que usa o tempo até o evento em vez de dicotomizar em 10 anos.

Parte III — Como funciona cada algoritmo

Regressão logística. Fronteira linear: cada variável recebe um peso, os pesos são somados e a soma vira probabilidade pela função logística. É o modelo mais transparente — o peso de cada gene é lido diretamente — e serve de referência: se um método complexo não a supera, a estrutura dos dados é essencialmente linear.

Random Forest. Centenas de árvores de decisão, cada uma treinada num reamostrado dos pacientes e enxergando apenas um sorteio das variáveis a cada divisão. A predição é a média. Essa dupla aleatoriedade descorrelaciona os erros: cada árvore erra de um jeito diferente e a média cancela boa parte do ruído. Captura interações sem que se precise declará-las.

Extra Trees. Variante em que os pontos de corte são sorteados em vez de otimizados. Mais aleatoriedade aumenta o viés de cada árvore mas reduz ainda mais a variância do conjunto — por isso vai bem quando há muitas variáveis de sinal fraco, como os 489 genes (é o segundo melhor no conjunto combinado).

Gradient Boosting. As árvores são construídas em sequência, e não em paralelo: cada nova árvore é treinada para corrigir o erro residual das anteriores e entra com peso pequeno (taxa de aprendizado). O ajuste é incremental e focado nos casos ainda mal previstos. Ganha precisão, mas é mais sensível a sobreajuste do que a floresta.

XGBoost. Gradient boosting com freios: regularização explícita dos pesos das folhas, poda por ganho mínimo, amostragem de linhas e colunas a cada rodada e busca de cortes por histograma. É o que permite rodar boosting em 500 colunas sem decorar o treino — e por isso lidera o conjunto combinado.

Rede neural (MLP). As variáveis passam por uma camada oculta com ativação não-linear e depois por uma saída logística, aprendendo combinações que nenhuma variável expressa isoladamente. Precisa de padronização e de regularização forte (o termo de penalização foi ajustado por busca). Lidera no conjunto clínico e é o **pior** nos genes — sinal claro de que, com 489

entradas e 1.560 casos, ela decora em vez de generalizar.

Parte IV — A plataforma nova

- **Seletor de conjunto** (clínico / genes / combinado) e **seletor de algoritmo**, com AUC e IC 95% de cada opção visíveis no momento da escolha.
- **Gráfico de curvas ROC** sobrepostas para os algoritmos do conjunto ativo, em SVG desenhado no navegador. A curva do algoritmo selecionado fica em destaque; a legenda liga e desliga cada uma; a diagonal marca o acaso.
- **Seção explicativa** com o funcionamento de cada algoritmo e sua métrica no conjunto ativo.
- **Tabela completa** dos 16 modelos, com a linha ativa destacada.
- **Estados de indisponibilidade explícitos** substituem os números falsos apontados na auditoria.

Todos os modelos rodam no navegador — inclusive as florestas (150 árvores) e o XGBoost (até 350 árvores), exportados como arrays compactos de nós. Testei os 16 em Node contra o scikit-learn: **erro máximo de 3×10^{-6}** .

Dois bugs meus, encontrados e corrigidos no processo

1. **Laço infinito na travessia das árvores.** As folhas do sklearn usam `feature = -2`, não `-1`; meu teste de parada nunca disparava. Diagnosticuei com `py-spy` depois de o processo travar 13 minutos. A condição correta é `children_left == -1`.
2. **XGBoost com erro de $4,8 \times 10^{-2}$.** O `base_score` real não é 0,5 e sim a prevalência ($\approx 0,45$); e os limiares precisam da precisão de float32 do booster. Após corrigir ambos, o erro caiu para $8,4 \times 10^{-8}$.

Publicação

`_headers` e `wrangler.toml` estão no pacote. O bundle tem 2,1 MB (os modelos de árvore são o grosso), servido comprimido pelo Cloudflare.

```
cd plataforma2_multialgoritmo
wrangler pages deploy . --project-name painel-metabric
```

Mantenho a recomendação de publicar atrás do Cloudflare Access, restrito ao domínio institucional.